

杭州电子科技大学信息工程学院

毕业设计（论文）开题报告

题 目	基于 Unity 的动作游戏与编辑器工具的设计与实现
系	计算机系
专 业	软件工程
姓 名	邬锦飞
班 级	软件工程一班
学 号	259400232
指导教师	王欣

一、综述本课题国内外研究动态，说明选题的依据和意义

1.1 国内研究动态

动作游戏的内容生产高度依赖连招与判定参数的反复调校。国内游戏公司在动作类、ARPG 类项目的研发中，已普遍采用“有限状态机 + 数据驱动 + 自研编辑器工具链”的生产方式，并在实践中沉淀了连招配置表、动作取消窗口、判定帧窗口、动画事件中继等工程经验，使连招与战斗参数的生产逐步从代码硬编码转向策划可视化配置。工具链是否好用，直接影响内容产能与调参效率。

在学术与毕业设计层面，国内相关研究多集中于角色动画控制、有限状态机、行为树 AI、Unity 组件开发等单点技术；以“使用 Unity 实现一款××游戏”为题的毕业设计数量众多，其重心通常在玩法实现与功能完整度上，编辑器工具多以附属功能的形式出现，缺少对工具自身设计方法（配置对象如何建模、配置粒度如何划分、配置错误如何检查、工具如何扩展）的讨论。

具体到连招配置这一环节，公开资料多为商业插件附带的通用技能系统，或直接以配置表形式存在，面向“每段连招的动画片段、攻击判定起止时间、伤害值、连招窗口时间、位移参数”的可视化配置与配置一致性校验，缺少可复现、可验证的设计方案。与此同时，大量学生项目与小型团队仍把连招参数与判定逻辑直接写在角

色控制器里：修改一次参数需要改动代码并重新编译，非程序人员无法参与调优，参数之间的约束关系（判定时间与动画时长、连招窗口与动画进度、位移与动画表现）没有任何检查手段，错误往往只能在运行时以“连招接不上”“挥空却打中”“判定不生效”等形式暴露。因此，面向动作游戏连招内容的可视化配置工具具有明确的实践价值。

1.2 国外研究动态

国外在引擎生态与工具化方面起步更早。Unity 官方提供了完整的编辑器扩展接口（EditorWindow、CustomEditor、SerializedObject、ReorderableList、AssetDatabase，以及 Handles 与 Scene 视图回调等），使团队内部工具可以直接嵌入编辑器 workflow；Unity 官方学习平台也提供了编辑器脚本开发的入门与最佳实践资料。在行业实践中，动作游戏的技能与连招编辑器是常见的自研工具，技术社区有关于 ACT 技能编辑器制作经验的公开分享，其共同点是“把时间轴上的判定、动画与表现做成可视化配置”。

在方法与理论层面，Robert Nystrom 在《游戏编程模式》中系统论述了组件模式、状态模式、观察者模式在游戏开发中的应用，为“数据驱动 + 组件装配”的设计提供了依据；Jason Gregory 在《Game Engine Architecture》中对角色动画系统、混合树、Root Motion 与连续碰撞检测作了完整阐述，是判定与动画时序设计的重要参考。

综合来看，国外的研究与实践提供了成熟的配置化范式，但其对象多为通用引擎能力与商业插件，配置项粒度较粗、概念层次较多；面向中小规模项目、聚焦“连招段参数 + 帧级时序 + 编辑期配置校验”的可复现设计与验证方案仍然较少。这正是本课题的切入点。

1.3 选题依据与意义

本课题的研究对象是 Unity 编辑器扩展工具，具体为面向动作游戏的连招配置编辑器；同时实现一个可运行的动作游戏 Demo，作为验证该工具可用性的验证平台。游戏玩法本身不作为研究内容，其在课题中提供的是真实的配置对象与验证场景。

选题依据来自这一工作场景的具体矛盾：连招与攻击参数具有时序性强、相互约束多、出错不报错的特点。以代码硬编码或裸 Inspector 编辑的方式组织这类数据存在三个问题：修改成本高，任何调参都需要改动代码并重新验证；协作门槛高，非程序人员难以参与；约束关系无法表达，配置错误只能在运行时暴露且多为静默失效，定位成本高。

本课题的意义体现在三个层面。工程层面，把主观的连招手感拆解为可度量、可配置的参数（攻击判定起止时间、伤害值、连招窗口时间、位移参数等），并使编辑器成为这些参数的唯一入口；效率层面，通过“编辑器修改→资产→运行时生效”的链路，使连招内容的生产脱离代码修改；方法层面，把配置合法性的检查前移到编辑期，用工具手段控制数据驱动架构固有的“错误静默化”风险。

本课题的重点与特色在于：以“连招段”为配置单元，把动画片段、攻击判定时间区间、伤害、连招窗口与位移参数统一到一套可视化配置中，并提供编辑期的配置校验；运行时按同一份资产驱动动画播放、判定与位移，保证“编辑器里配的”和“游戏里跑的”一致。

二、研究的基本内容，拟解决的主要问题

2.1 基本内容

本课题的目标是：设计并实现一个基于 Unity 的动作游戏 Demo，包含角色移动、攻击连招、受击反馈和简单敌人 AI 等基础动作游戏要素；同时开发一个 Unity 编辑器扩展工具——连招配置编辑器，使策划与开发者能够通过可视化界面配置连招转换规则、攻击判定参数与动画事件，无需直接修改代码。

具体内容包括：连招以“段”为单位组织，工具支持连招步骤的添加、删除与排序，并支持配置段与段之间的转换规则（可接续的下一段、输入窗口与取消窗口）；每一段可配置动画片段、动画事件、攻击判定起始与结束时间、伤害值、连招窗口时间与位移参数；配置数据以 ScriptableObject 资产持久化，支持在编辑器内保存与加载；运行时从资产读取配置，驱动角色动画播放与攻击判定逻辑，形成“编辑器配置→运行时执行”的完整链路；工具对动画片段未赋值、时间区间非法等配置错误给出提示。动作游戏 Demo 提供真实的配置对象与验证场景，包含角色控制、连招战斗、受击反馈、敌人 AI 与界面显示。

2.1.1 功能模块图

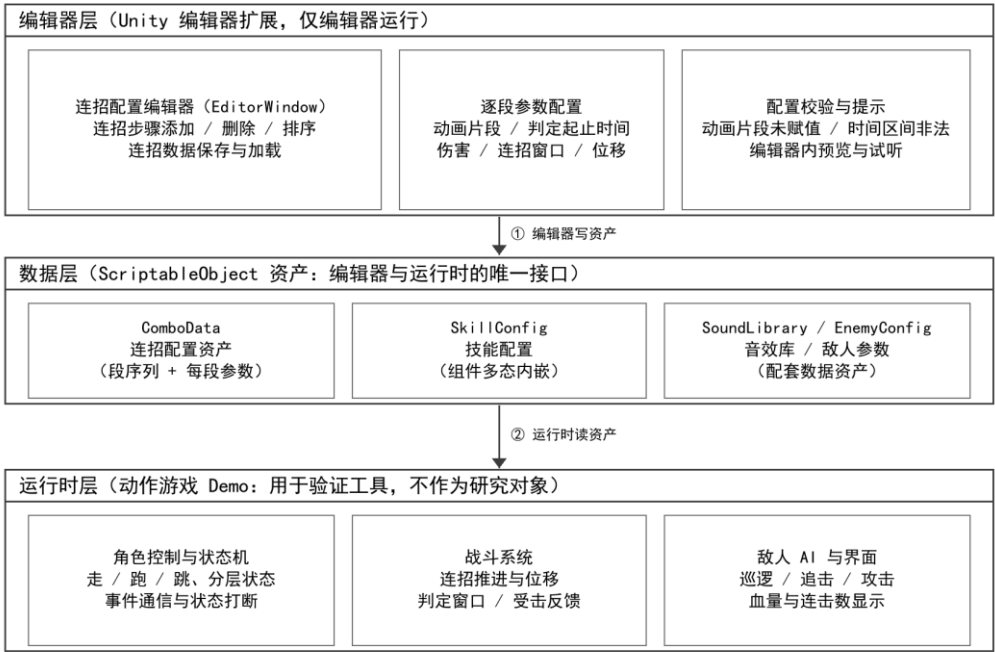


图 2-1 连招配置编辑器总体结构图

2.1.2 模块介绍

- （1）连招序列与转换规则配置模块。在编辑器窗口中列出当前连招的全部步骤，支持步骤的添加、删除与排序；同时配置段与段之间的转换规则，包括当前段可接续的下一段、连招窗口（输入下一段指令的有效时间区间）与取消窗口（允许用闪避等动作取消当前段的时间区间）。连招整体支持保存与加载，保证配置结构清晰、可反复调整。
- （2）逐段参数配置模块。每一段连招提供完整参数配置：动画片段（该段播放的动画）、动画事件（在该段指定时间点触发的表现事件，如打击音效与特效）、攻击判定起始/结束时间（判定窗口的开启与关闭时机）、伤害值、连招窗口时间与位移参数（如攻击前冲距离）。
- （3）配置校验与提示模块。在编辑器内对配置做静态检查，对动画片段未赋值、判定时间区间非法（起始晚于结束、超出动画时长）、连招窗口与判定窗口冲突等情形给出明确提示，把错误暴露在编辑阶段。
- （4）数据持久化与运行时对接模块。配置以 `ScriptableObject` 资产存储，编辑器负责读写；运行时按连招标识读取资产，驱动动画播放、判定窗口开关与位移。工具提供菜单按钮与快捷键两种配置重新加载入口，使编辑器修改后无需重新启动游戏即可验证效果。
- （5）角色控制与状态管理模块。基于 `Rigidbody` 的位移接口实现角色移动（走/跑切换）与跳跃，移动动画通过 `Animator` 混合树平滑过渡；采用分层状态机的思路，

把移动状态与攻击状态分离，状态之间通过事件通信实现打断（如攻击后摇可接闪避）。

（6）战斗系统模块。实现轻攻击连招（不少于三段），攻击命中后触发受击反馈（受击动画、击退、短暂硬直）；攻击判定采用 `Physics.OverlapSphere` 与 `Physics.OverlapCapsule` 实现球/胶囊体积检测（对应任务书要求的 `Capsule Collider` 与 `OverlapSphere` 方案），并对高速挥砍做帧间扫掠采样，判定窗口的开启与关闭由动画事件与配置时间共同驱动，避免漏判与误判。

（7）敌人 AI 模块。实现基础的巡逻与追击行为：敌人在巡逻状态下沿设定路径移动，发现玩家后进入追击，进入攻击范围后对玩家发起攻击；受击、僵直与死亡状态与攻击状态互斥，避免动画与行为冲突。

（8）界面模块。显示玩家血量与连招提示（如当前连击数），用于直观呈现连招推进结果与战斗状态。

2.1.3 预期成果

- （1）连招配置编辑器一个，包含连招步骤管理、逐段参数配置、配置校验与提示、数据保存与加载；
- （2）可运行于 PC 端的动作游戏 Demo 一套（角色控制与跳跃、三段以上连招、受击反馈、敌人巡逻与追击、血量与连击数界面）；
- （3）连招配置数据资产（`ScriptableObject`）与运行时对接链路；
- （4）连招触发、命中判定、配置保存与加载的功能测试记录，以及帧率测试数据；
- （5）毕业设计论文一篇，以及系统演示视频与答辩材料。

2.1.4 技术指标与考核标准

- （1）开发环境：Unity 2021.3 LTS 或更高版本、URP 渲染管线、C#脚本语言。本课题实际使用团结引擎 1.9.2，其内核基于 Unity 2022.3 LTS，满足任务书“Unity 2021.3 LTS 或更高版本”的要求（团结引擎为 Unity 面向国内发行的引擎版本，具体口径拟请指导教师确认）。
- （2）编辑器工具置于 `Assets/Editor` 目录下，确保编辑器代码不被打包进最终构建。
- （3）实现不少于 3 段连招的完整配置与运行，连招衔接流畅，无明显卡顿或跳帧。
- （4）攻击判定采用 `Physics.OverlapSphere` 与 `Physics.OverlapCapsule` 进行球/胶囊体积检测（任务书指定的 `Capsule Collider` 与 `OverlapSphere` 方案），并对高速挥砍做帧间扫掠采样，避免漏判与误判。
- （5）连招配置编辑器具备基本的错误校验能力，如动画片段未赋值、判定时间区间非法时给出提示。
- （6）游戏 Demo 可在 PC 端运行，帧率稳定在 60FPS 以上。

2.2 拟解决的主要问题

2.2.1 功能结构设计

系统按“编辑器层—数据层—运行时层”三层结构组织：编辑器层负责连招配置的编辑与校验；数据层以 `ScriptableObject` 资产承载配置，是编辑器与运行时之间唯一的接口；运行时层按资产驱动角色动画、判定与位移。层与层之间不存在直接调用，因此修改配置不需要重新编译运行时代码。

连招配置采用“段序列 + 每段参数”的数据结构：一条连招由若干有序的段组成，每段包含动画片段、判定起止时间、伤害值、连招窗口时间与位移参数。选择这一结构的原因在于，动作游戏的连招手感由“段与段之间的衔接规则”与“单段内部的时间关系”共同决定，把两者分开表达，才能在编辑器里同时支持整体排序与逐段微调。由此需要解决两个具体问题：一是段序列在编辑器中的增删与排序要保证索引与数据一致；二是判定与位移必须以动画的时间轴为基准，否则配置数值与实际表现会对不上。

时序结构上，本课题以动画时间（帧号或归一化进度）作为统一基准：判定窗口以起止时间表达，连招窗口以动画进度表达，帧事件用于触发音效等表现。这样处理的原因在于，动作游戏中判定、动画与反馈必须落在同一时间基准上，否则会出现“看到刀挥过却没有打中”或“打中了却没有音效”的现象。需要进一步解决判定时间与动画时长的约束关系、不同帧率素材下的换算，以及编辑器预览与运行时的触发一致性验证。

连招的业务规则在实现中需满足三条约束：一是连招窗口必须在动画的特定时间段内有效，超出窗口后连招中断并回到移动或待机状态；二是同一连招步骤在一次连招过程中不可被重复触发，直到连招结束或被重置；三是编辑器修改配置后，运行时通过菜单按钮或快捷键重新加载数据即可生效，不需要重新启动游戏。

2.2.2 框架搭建

编辑器代码不应进入运行时构建，配置数据的修改也应避免直接改写字段造成序列化丢失。拟采取的措施包括：将编辑器脚本置于 `Assets/Editor` 目录并与运行时脚本通过程序集定义文件隔离；配置写入统一经由 `SerializedObject` 等序列化接口完成并标记为已修改；为关键编辑操作提供撤销支持；编辑器状态通过会话状态保存，避免脚本重编译后丢失。

工具界面基于 `EditorWindow` 与 `IMGUI` 搭建，使用可重排列表管理连招步骤与每段参数，通过 `SerializedObject` 与 `SerializedProperty` 完成数据绑定与写回；调参过程辅以可视化支持，包括判定体积在 `Scene` 视图中的线框显示与拖拽手柄、动画时间轴与帧号显示、编辑器内音效试听，减少“改一次、跑一次”的往返。

2.2.3 数据库设计（数据资产层）

本课题为单机游戏客户端，不使用传统的关系数据库，本小节描述的是系统的数据资产层设计：全部可配置内容以 Unity 的 ScriptableObject 资产承载，运行时只读，编辑器统一经序列化接口写入，避免直接修改字段造成序列化丢失。主要数据资产及其字段如表 1 所示，其中连招配置资产是本次编辑器工具的主要编辑对象。

表 1 主要数据资产及其字段

数据资产	主要字段	说明
ComboData 连招配置资产	连招段序列、每段动画片段、判定起止时间、伤害值、连招窗口时间、位移参数	本次编辑器工具的主要编辑对象，运行时据此驱动动画、判定与位移
SkillConfig 技能配置	技能标识、组件列表（播放动画、判定窗口、伤害、命中反馈、无敌帧）	以组件化方式描述单段技能行为，作为连招段实现的扩展载体
SoundLibrary 音效库	音效标识、音频片段	打击音效与脚步音效的查表播放与随机变体
EnemyConfig 敌人配置	血量、韧性、移动速度、检测与攻击范围、攻击间隔	敌人 AI 的参数配置，供巡逻与追击行为使用

2.2.4 安全性

单机游戏的安全性主要体现在工程安全、数据正确性与运行健壮性三个方面。工程安全方面，资产修改统一经由序列化接口完成，编辑器侧的试听与预览使用临时对象，不写入资产。

数据正确性方面，本课题在编辑期建立一组校验规则，并保证规则与运行时语义一致，包括引用完整性（动画片段、判定参数是否已赋值）、时序合法性（判定起止时间是否落在动画时长内、连招窗口是否与判定窗口冲突）、外部一致性（动画片段是否与角色动画资源匹配）与结构约束（连招段序号与位移参数是否合法）。这一部分是本课题的重点：数据驱动架构把错误从编译期推迟到运行期，且多数配置错误不会抛出异常，只表现为功能不生效，定位成本高；把校验前移到编辑期，可以直接降低这类风险。

运行健壮性方面，连招或攻击被打断时统一执行清理，避免判定窗口、无敌状态与位移残留到后续行为；帧帧并发触发时按统一策略恢复时间缩放；运行时输出经日志门控控制，便于定位问题。

2.2.5 功能性测试

测试分两部分。功能测试覆盖游戏侧与工具侧：游戏侧包括连招触发与推进（连续输入是否按预期进入下一段、窗口外输入是否被忽略、同一步骤是否会被重复触发、超出连招窗口后是否正确中断）、命中判定（是否存在漏判与误判）、受击反馈与敌人 AI 状态转移；工具侧包括连招步骤的增删排序、配置保存与加载、配置校验提示是否触发，以及通过菜单按钮或快捷键重新加载配置后运行时能否立即生效。

自动化与指标测试包括：EditMode 单元测试（在 Unity 编辑环境中运行的自动化测试，不进入游戏构建包）覆盖数据层与连招推进逻辑；命令行编译门禁保证提交代码可编译；性能测试使用 Profiler 统计 PC 端运行帧率，验证是否稳定在 60FPS 以上；配置正确性测试采用错误注入的方式，人为构造若干类错误配置，统计工具的检出情况与定位耗时，并与无校验的人工排查作对照。

2.2.6 其他关键技术问题

动画事件与判定窗口的同步问题（动画事件触发时机与配置时间不一致会导致判定错位）；连招窗口的手感调校（窗口过窄导致连招接不上、过宽导致乱按也能连）；配置改动后的运行时重新加载策略；帧帧并发触发时的恢复策略；高频对象（音效播放、伤害数字）的对象池复用与状态重置；编辑器长时间运行下的性能与状态一致性。

三、研究步骤、方法及措施

3.1 研究步骤

（1）需求分析与资料收集：查阅任务书、引擎官方文档与编辑器开发相关资料，明确连招配置工具的功能边界与配置对象范围。本阶段已完成。

（2）动作游戏 Demo 核心闭环：实现角色控制、攻击连招、受击反馈、敌人状态机、音效与胜负结算，为工具提供真实的配置对象与验证场景。本阶段已完成。

（3）数据层与编辑器工具主体：定义连招与技能配置的数据资产，实现编辑器窗口、参数配置界面与配置校验功能。本阶段已完成主体，正在按任务书口径扩展连招配置能力。

（4）连招配置编辑器完善：实现连招步骤的添加、删除与排序，补齐每段动画片段、判定起止时间、伤害值、连招窗口时间与位移参数的配置入口。本阶段正在进行。

（5）任务书要求的补充功能：角色跳跃、敌人巡逻、连击数界面显示。本阶段正在进行。

（6）校验与测试：完善编辑器配置校验规则，补充 EditMode 单元测试，开展连招触发、命中判定、配置保存加载与帧率测试。本阶段计划进行。

（7）论文撰写与答辩准备：论文按任务书给定的六章结构组织，即绪论、相关技术、系统分析、系统设计、系统实现、系统测试，最后是结论；按此结构整理设计过程、关键技术、测试数据与演示材料。本阶段计划进行。

说明：以上进度状态按实际完成情况标注。本课题在开题阶段已完成动作游戏 Demo 的核心闭环与编辑器工具主体，开题后的工作重点为连招配置能力的补齐、任务书要求的补充功能、配置校验与测试，以及论文撰写。

3.2 研究方法

- （1）文献与资料研究法。以任务书、Unity 官方文档、编辑器开发专著与技术社区经验分享为依据，确定设计取舍并在报告中说明比较结果。
- （2）原型迭代法。先打通“配置数据—运行时读取—表现反馈”的最小闭环，再逐步扩展连招段数、判定形状与配置项，每一步都在运行环境中验证。
- （3）对照实验法。对关键方案设置对照组：连招参数使用编辑器配置与直接修改代码的人工方式对比，比较修改步骤数、所需时间与出错概率；判定方案对比以角色为中心的球形检测与贴合武器节点的体积检测加帧间扫掠采样。
- （4）单元测试与性能测试法。对数据层与连招推进逻辑编写 EditMode 自动化测试，防止修改引入回归缺陷；使用 Profiler 统计运行帧率，验证性能指标。

3.3 技术路线与措施

系统的数据流为：连招配置编辑器（编辑连招步骤与每段参数，写入资产）→ 数据层（连招配置、技能配置、音效库等 ScriptableObject 资产）→ 运行时（读取资产，驱动角色动画播放、判定窗口开关与位移，并向界面与音效反馈）。编辑器层与运行时层之间不存在直接调用，二者唯一的接口是资产本身，因此配置修改不需要重新编译运行时代码。

主要措施包括：编辑器脚本统一置于 Assets/Editor 目录并与运行时程序集隔离；使用 Git 进行版本管理；使用命令行编译检查与单元测试作为提交前的门禁；对关键配置项建立校验规则并集中提示；对高频创建销毁的对象使用对象池；使用日志门控控制运行时输出；使用 Profiler 定位性能瓶颈。

四、研究工作进度

本课题起止日期为 2026 年 6 月 10 日至 2027 年 1 月 12 日，进度安排与任务书一致，见下表。

序号	时间	内容
1	2026.06.10 - 2026.06.19	课题申报
2	2026.06.20 - 2026.06.23	学生选题

序号	时间	内容
3	2026.06.24 - 2026.06.26	确认选题
4	2026.06.27 - 2026.07.06	任务书
5	2026.07.07 - 2026.09.21	开题答辩
6	2026.09.22 - 2026.11.18	软件编码、论文撰写
7	2026.10.21 - 2026.10.27	中期检查
8	2026.11.19 - 2026.11.25	院内互评
9	2026.11.30 - 2026.12.4	校内评阅
10	2026.12.7 - 2026.12.18	校外盲审
11	2026.12.22 - 2026.12.31	毕业答辩
12	2027.01.03 - 2027.01.12	成绩评议，材料归档

五、主要参考文献

- [1] 张寿昆. Unity 编辑器开发与拓展[M]. 北京: 清华大学出版社, 2024.
- [2] NYSTROM R. 游戏编程模式[M]. 北京: 人民邮电出版社, 2016.
- [3] Gordon. ACT 技能编辑器的制作经验分享[EB/OL]. 侑虎科技, 2018.
- [4] Unity Technologies. Introduction to editor scripting[EB/OL]. Unity Learn, 2025.
- [5] Unity Technologies. Unity manual: scripting concepts[EB/OL]. Unity Documentation, 2025.
- [6] GREGORY J. Game engine architecture[M]. 3rd ed. Boca Raton: CRC Press, 2018.
- [7] 胡凯宁. 基于 Unity3D 的像素休闲游戏的设计与实现[J]. 电脑编程技巧与维护, 2026(4): 147-149.
- [8] 巩云飞. 基于 Unity3D 的游戏设计与实现[J]. 现代信息科技, 2025, 9(8): 89-92.
- [9] 路宜松. 基于 Unity 引擎的 2D 角色扮演游戏的设计与实现[D]. 沈阳: 沈阳理工大学, 2021.
- [10] 沈士钊. 基于 Unity3D 引擎的三维角色扮演游戏设计与实现[D]. 武汉: 华中科技大学, 2017.
- [11] 王巍, 高德远. 有限状态机设计策略[J]. 计算机工程与应用, 1999(7): 54-55.
- [12] 马晓萍, 轩莹莹. 基于 Unity 3D 的电子游戏设计与实现[J]. 电子技术, 2024, 53(6): 75-79.